

▼ Part 0: Google Cloud Authentication

Colab requires us to verify our Google account before it allows us to use the high-speed gcloud pipeline to download the dataset. Run this cell and click through the standard Google permissions popup.

```
from google.colab import auth
auth.authenticate_user()
print("✅ Authenticated successfully! You can now run the data download cell.")
```

✅ Authenticated successfully! You can now run the data download cell.

▼ Part 1: Pulling Satellite Data from Google Cloud

We need to bypass standard slow downloads and rip the gold-standard Sen1Floods11 hand-labeled dataset straight from Google Cloud Storage into our Colab instance.

```
# Create the exact directory structure
!mkdir -p /content/sen1floods11/HandLabeled

# Use the fast gcloud storage command to pull the folder recursively
!gcloud storage cp -r gs://sen1floods11/v1.1/data/flood_events/HandLabeled/* /content/sen1floods11/HandLabeled/
```

Average throughput: 54.9MB/s

To take a quick anonymous survey, run:
\$ gcloud survey

🌸 Part 2: Installing Dependencies Colab has PyTorch pre-installed, but we need rasterio for reading .tif satellite files and segmentation-models-pytorch to easily build our U-Net.

```
!pip install segmentation-models-pytorch rasterio
```

```
Collecting segmentation-models-pytorch
  Downloading segmentation_models_pytorch-0.5.0-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: rasterio in /usr/local/lib/python3.12/dist-packages (1.5.0)
Requirement already satisfied: huggingface-hub>=0.24 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.0.0)
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.0.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (11.3.0)
Requirement already satisfied: safetensors>=0.3.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (0.4.5)
Requirement already satisfied: timm>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (1.0.28)
Requirement already satisfied: torch>=1.8 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (2.11.0)
Requirement already satisfied: torchvision>=0.9 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (0.15.2)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist-packages (from segmentation-models-pytorch) (4.67.1)
Requirement already satisfied: affine in /usr/local/lib/python3.12/dist-packages (from rasterio) (2.4.0)
Requirement already satisfied: attrs in /usr/local/lib/python3.12/dist-packages (from rasterio) (26.1.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from rasterio) (2026.6.17)
Requirement already satisfied: click!=8.2.*,>=4.0 in /usr/local/lib/python3.12/dist-packages (from rasterio) (8.4.2)
Requirement already satisfied: cligj>=0.5 in /usr/local/lib/python3.12/dist-packages (from rasterio) (0.7.2)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-packages (from rasterio) (3.3.2)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (3.10.0)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (2025.10.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (0.23.0)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (6.0.2)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24->segmentation-models-pytorch) (4.13.2)
Requirement already satisfied: setuptools>=82 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (75.8.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch) (3.1.4)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch)
Requirement already satisfied: nvidia-cusparse-cu12==12.8.9.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cuda-bindings]<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8->segmentation-models-pytorch))
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (4.8.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (1.0.7)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (3.10)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.24->segmentation-models-pytorch) (0.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8->segmentation-models-pytorch) (3.2.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.8->segmentation-models-pytorch) (3.0.2)
Downloading segmentation_models_pytorch-0.5.0-py3-none-any.whl (154 kB)
154.8/154.8 kB 6.3 MB/s eta 0:00:00
Installing collected packages: segmentation-models-pytorch
Successfully installed segmentation-models-pytorch-0.5.0
```

🌸 Part 3: Environment Setup & Imports Importing our core libraries and ensuring PyTorch is hooked up to the free T4 GPU so our model trains in minutes instead of days.

```
import os
import glob
import torch
import rasterio
import numpy as np
import matplotlib.pyplot as plt
```

```

from torch.utils.data import Dataset, DataLoader
import segmentation_models_pytorch as smp
import torch.nn as nn
import torch.optim as optim

# Check for GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"🔥 Currently running on device: {device}")

```

🔥 Currently running on device: cuda

📁 Part 4: Building the SAR Dataset Loader Sentinel-1 SAR radar imagery uses two polarization bands (VV and VH). This custom dataset class pairs those 2-channel images with their corresponding binary flood masks (1 = Flood, 0 = Dry Land).

```

class Sen1FloodsDataset(Dataset):
    def __init__(self, data_dir):
        # 1. Recursively search through all subfolders for the radar images
        self.image_paths = sorted(glob.glob(os.path.join(data_dir, '**', '*_S1Hand.tif'), recursive=True))

        # 2. The masks are in a different folder with a different suffix.
        # We replace both the folder name ('S1Hand' -> 'LabelHand') and the file suffix.
        self.mask_paths = [path.replace('S1Hand', 'LabelHand') for path in self.image_paths]

        print(f"✅ Found {len(self.image_paths)} image/mask pairs.")

        # A quick safety check just in case the download failed entirely
        if len(self.image_paths) == 0:
            print("🔥 ERROR: Still 0 files found. Let's check what is actually in the folder:")
            os.system(f"ls -R {data_dir} | head -n 20")

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        # 1. Read SAR Image (2 bands: VV and VH)
        with rasterio.open(self.image_paths[idx]) as src:
            image = src.read()
            image = np.nan_to_num(image)
            image = np.clip(image, -30, 0)
            image = (image + 30) / 30.0

        # 2. Read Ground Truth Mask
        with rasterio.open(self.mask_paths[idx]) as src:
            mask = src.read(1)
            mask = np.where(mask == 1, 1.0, 0.0)

        # 3. Convert to PyTorch Tensors
        image_tensor = torch.tensor(image, dtype=torch.float32)
        mask_tensor = torch.tensor(mask, dtype=torch.float32).unsqueeze(0)

        return image_tensor, mask_tensor

# Initialize the dataset and loader
from torch.utils.data import random_split

# Initialize the dataset
data_directory = '/content/sen1floods11/HandLabeled/'
full_dataset = Sen1FloodsDataset(data_directory)

# Split into 80% training and 20% validation
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size

train_dataset, val_dataset = random_split(
    full_dataset,
    [train_size, val_size]
)

# Training DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=4,
    shuffle=True
)

```

```
# Validation DataLoader
val_loader = DataLoader(
    val_dataset,
    batch_size=4,
    shuffle=False
)

print(f"Training samples: {len(train_dataset)}")
print(f"Validation samples: {len(val_dataset)}")
```

✓ Found 446 image/mask pairs.
 Training samples: 356
 Validation samples: 90

 **Part 5 (Upgraded): U-Net with Weighted Loss** We are using a U-Net with a ResNet34 backbone. To combat "Class Imbalance" (where 95% of the map is dry land), we are adding a pos_weight to our Loss Function. This forces the model to pay 10x more attention to the rare flood pixels, preventing it from just lazily guessing "Dry Land" everywhere.

```
# 1. Initialize the Segmentation Model
model = smp.Unet(
    encoder_name="resnet34",
    encoder_weights=None,
    in_channels=2,
    classes=1,
)

model = model.to(device)

# 2. Add the "Bounty" (Positive Weight)
# This tells the model: Missing a flood pixel is 10x worse than missing a dry pixel
weight_multiplier = torch.tensor([10.0]).to(device)

# 3. Initialize the Weighted Loss Function and Optimizer
criterion = nn.BCEWithLogitsLoss(pos_weight=weight_multiplier)
optimizer = optim.Adam(model.parameters(), lr=0.0005)
print("🏆 Model and Weighted Loss Function initialized successfully!")
```

🏆 Model and Weighted Loss Function initialized successfully!

 **Part 6 (Upgraded): Advanced Training Loop** We are bumping the training up to 30 epochs. We are also introducing a ReduceLROnPlateau scheduler. If the model stops improving (e.g., gets stuck looking at the black border), the scheduler dynamically reduces the learning rate to help the model zero in on the finer details of the actual floodwater.

```
# 1. Reset the optimizer to clear out its old memory
optimizer = optim.Adam(model.parameters(), lr=0.0001)

# 2. Add the Scheduler (Removed 'verbose=True' to fix PyTorch TypeError)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=3, factor=0.5)

def iou_score(pred, target, threshold=0.5):
    pred = (torch.sigmoid(pred) > threshold).float()
    intersection = (pred * target).sum()
    union = pred.sum() + target.sum() - intersection
    return (intersection + 1e-6) / (union + 1e-6)

def dice_score(pred, target, threshold=0.5):
    pred = (torch.sigmoid(pred) > threshold).float()
    intersection = (pred * target).sum()
    return (2 * intersection + 1e-6) / (pred.sum() + target.sum() + 1e-6)

def train_model_advanced(model, train_loader, val_loader, epochs=30):
    print("🚀 Initiating deep-dive training sequence...")

    best_loss = float('inf')

    for epoch in range(epochs):

        # ===== TRAIN =====
        model.train()
        train_loss = 0
        train_iou = 0
        train_dice = 0

        for images, masks in train_loader:
            images = images.to(device)
            masks = masks.to(device)
```

```

optimizer.zero_grad()

outputs = model(images)
loss = criterion(outputs, masks)

loss.backward()
optimizer.step()

train_loss += loss.item()
train_iou += iou_score(outputs, masks).item()
train_dice += dice_score(outputs, masks).item()

avg_train_loss = train_loss / len(train_loader)
avg_train_iou = train_iou / len(train_loader)
avg_train_dice = train_dice / len(train_loader)

# ===== VALIDATION =====
model.eval()

val_loss = 0
val_iou = 0
val_dice = 0

with torch.no_grad():
    for images, masks in val_loader:
        images = images.to(device)
        masks = masks.to(device)

        outputs = model(images)
        loss = criterion(outputs, masks)

        val_loss += loss.item()
        val_iou += iou_score(outputs, masks).item()
        val_dice += dice_score(outputs, masks).item()

avg_val_loss = val_loss / len(val_loader)
avg_val_iou = val_iou / len(val_loader)
avg_val_dice = val_dice / len(val_loader)

scheduler.step(avg_val_loss)

if avg_val_loss < best_loss:
    best_loss = avg_val_loss
    torch.save(model.state_dict(), "flood_unet_resnet34_best.pth")
    print("✅ Best model saved!")

current_lr = optimizer.param_groups[0]['lr']

print(
    f"Epoch {epoch+1:02d}/{epochs} | "
    f"Train Loss: {avg_train_loss:.4f} | "
    f"Train IoU: {avg_train_iou:.4f} | "
    f"Train Dice: {avg_train_dice:.4f} | "
    f"Val Loss: {avg_val_loss:.4f} | "
    f"Val IoU: {avg_val_iou:.4f} | "
    f"Val Dice: {avg_val_dice:.4f} | "
    f"LR: {current_lr:.6f}"
)

# Execute the advanced training! Grab a coffee, this will take a few minutes.
train_model_advanced(model, train_loader, val_loader, epochs=30)

```

🚀 Initiating deep-dive training sequence...

✅ Best model saved!

Epoch 01/30 | Train Loss: 0.5134 | Train IoU: 0.3633 | Train Dice: 0.4936 | Val Loss: 0.5574 | Val IoU: 0.3759 | Val Dice: 0.508

✅ Best model saved!

Epoch 02/30 | Train Loss: 0.4905 | Train IoU: 0.3638 | Train Dice: 0.4975 | Val Loss: 0.5518 | Val IoU: 0.4916 | Val Dice: 0.621

Epoch 03/30 | Train Loss: 0.4709 | Train IoU: 0.3593 | Train Dice: 0.4914 | Val Loss: 0.5623 | Val IoU: 0.3450 | Val Dice: 0.477

✅ Best model saved!

Epoch 04/30 | Train Loss: 0.5142 | Train IoU: 0.3533 | Train Dice: 0.4859 | Val Loss: 0.5211 | Val IoU: 0.4713 | Val Dice: 0.601

✅ Best model saved!

Epoch 05/30 | Train Loss: 0.4940 | Train IoU: 0.3517 | Train Dice: 0.4846 | Val Loss: 0.5040 | Val IoU: 0.4558 | Val Dice: 0.587

Epoch 06/30 | Train Loss: 0.4939 | Train IoU: 0.3475 | Train Dice: 0.4792 | Val Loss: 0.5280 | Val IoU: 0.4718 | Val Dice: 0.603

Epoch 07/30 | Train Loss: 0.4722 | Train IoU: 0.3549 | Train Dice: 0.4884 | Val Loss: 0.5299 | Val IoU: 0.4768 | Val Dice: 0.607

Epoch 08/30 | Train Loss: 0.4682 | Train IoU: 0.3618 | Train Dice: 0.4978 | Val Loss: 0.5369 | Val IoU: 0.4739 | Val Dice: 0.605

✅ Best model saved!

Epoch 09/30 | Train Loss: 0.4819 | Train IoU: 0.3601 | Train Dice: 0.4931 | Val Loss: 0.4816 | Val IoU: 0.4341 | Val Dice: 0.564

Epoch 10/30	Train Loss: 0.4792	Train IoU: 0.3655	Train Dice: 0.5046	Val Loss: 0.5887	Val IoU: 0.4417	Val Dice: 0.573
Epoch 11/30	Train Loss: 0.5195	Train IoU: 0.3508	Train Dice: 0.4816	Val Loss: 0.5165	Val IoU: 0.4457	Val Dice: 0.576
Epoch 12/30	Train Loss: 0.4813	Train IoU: 0.3601	Train Dice: 0.4979	Val Loss: 0.5240	Val IoU: 0.4522	Val Dice: 0.585
Epoch 13/30	Train Loss: 0.4589	Train IoU: 0.3808	Train Dice: 0.5172	Val Loss: 0.6598	Val IoU: 0.5016	Val Dice: 0.630
Epoch 14/30	Train Loss: 0.4577	Train IoU: 0.3653	Train Dice: 0.5007	Val Loss: 0.6348	Val IoU: 0.5113	Val Dice: 0.638
Epoch 15/30	Train Loss: 0.4386	Train IoU: 0.3742	Train Dice: 0.5107	Val Loss: 0.5076	Val IoU: 0.4902	Val Dice: 0.619
Epoch 16/30	Train Loss: 0.4701	Train IoU: 0.3741	Train Dice: 0.5116	Val Loss: 0.6615	Val IoU: 0.5192	Val Dice: 0.645
Epoch 17/30	Train Loss: 0.4596	Train IoU: 0.3657	Train Dice: 0.4994	Val Loss: 0.5064	Val IoU: 0.4896	Val Dice: 0.619
✅ Best model saved!						
Epoch 18/30	Train Loss: 0.4410	Train IoU: 0.3751	Train Dice: 0.5132	Val Loss: 0.4770	Val IoU: 0.4693	Val Dice: 0.599
Epoch 19/30	Train Loss: 0.4290	Train IoU: 0.4019	Train Dice: 0.5447	Val Loss: 0.4871	Val IoU: 0.4786	Val Dice: 0.608
Epoch 20/30	Train Loss: 0.4472	Train IoU: 0.3774	Train Dice: 0.5109	Val Loss: 0.5225	Val IoU: 0.4888	Val Dice: 0.618
Epoch 21/30	Train Loss: 0.4251	Train IoU: 0.3986	Train Dice: 0.5404	Val Loss: 0.5213	Val IoU: 0.5021	Val Dice: 0.630
Epoch 22/30	Train Loss: 0.4415	Train IoU: 0.3478	Train Dice: 0.4821	Val Loss: 0.4873	Val IoU: 0.4706	Val Dice: 0.601
Epoch 23/30	Train Loss: 0.4470	Train IoU: 0.3726	Train Dice: 0.5075	Val Loss: 0.4832	Val IoU: 0.4840	Val Dice: 0.613
Epoch 24/30	Train Loss: 0.4295	Train IoU: 0.3891	Train Dice: 0.5288	Val Loss: 0.4984	Val IoU: 0.4848	Val Dice: 0.614
Epoch 25/30	Train Loss: 0.4231	Train IoU: 0.3834	Train Dice: 0.5248	Val Loss: 0.4863	Val IoU: 0.4849	Val Dice: 0.615
✅ Best model saved!						
Epoch 26/30	Train Loss: 0.4209	Train IoU: 0.3789	Train Dice: 0.5138	Val Loss: 0.4738	Val IoU: 0.4609	Val Dice: 0.591
Epoch 27/30	Train Loss: 0.4252	Train IoU: 0.3778	Train Dice: 0.5154	Val Loss: 0.4960	Val IoU: 0.4858	Val Dice: 0.616
Epoch 28/30	Train Loss: 0.4223	Train IoU: 0.3807	Train Dice: 0.5153	Val Loss: 0.5135	Val IoU: 0.4896	Val Dice: 0.619
Epoch 29/30	Train Loss: 0.4133	Train IoU: 0.3815	Train Dice: 0.5183	Val Loss: 0.4764	Val IoU: 0.4661	Val Dice: 0.596
Epoch 30/30	Train Loss: 0.4130	Train IoU: 0.3847	Train Dice: 0.5240	Val Loss: 0.4818	Val IoU: 0.4666	Val Dice: 0.596

 Part 7: Visualizing the Predictions Let's pull a random satellite image, ask the trained model to predict where the flood zones are, and compare it visually against the actual hand-labeled truth.

```
def visualize_prediction(model, dataset, index=0):
    model.eval()

    image, true_mask = dataset[index]
    image_input = image.unsqueeze(0).to(device)

    with torch.no_grad():
        raw_prediction = model(image_input)
        prob_mask = torch.sigmoid(raw_prediction)
        # Threshold at 0.5 to make a definitive "Flood" or "No Flood" prediction
        predicted_mask = (prob_mask > 0.5).float()

    # Move tensors back to CPU for plotting
    image_plot = image.cpu().numpy()
    true_mask_plot = true_mask.cpu().numpy().squeeze()
    predicted_mask_plot = predicted_mask.cpu().numpy().squeeze()

    # Create the aesthetic visualization
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))

    axes[0].imshow(image_plot[0], cmap='gray')
    axes[0].set_title("📡 Original SAR Image (VV Band)", fontsize=14)
    axes[0].axis('off')

    axes[1].imshow(true_mask_plot, cmap='Blues')
    axes[1].set_title("🌊 Actual Hand-Labeled Flood", fontsize=14)
    axes[1].axis('off')

    axes[2].imshow(predicted_mask_plot, cmap='Blues')
    axes[2].set_title("🧠 Deep Learning Prediction", fontsize=14)
    axes[2].axis('off')

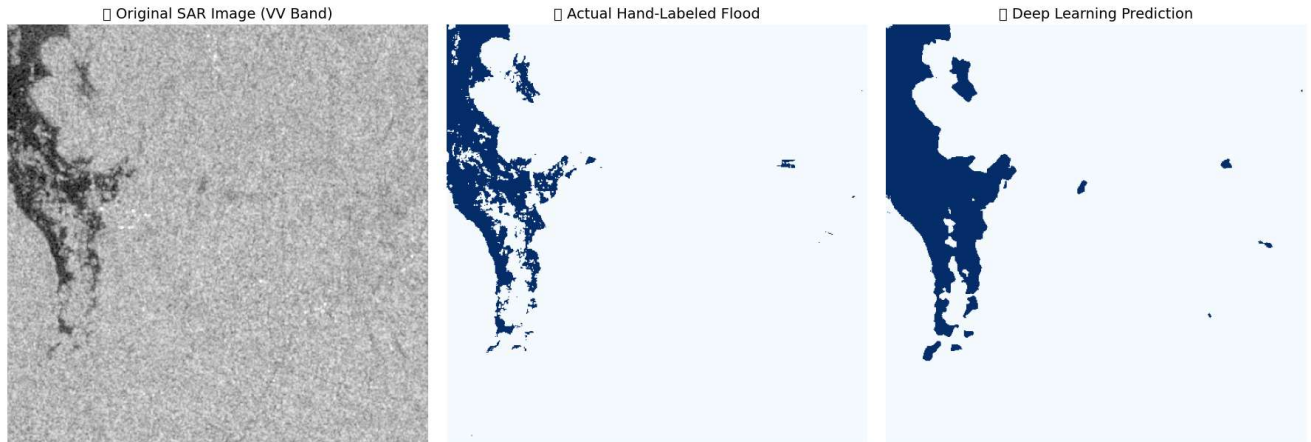
    plt.tight_layout()
    plt.show()

# Visualize the 10th image in our dataset
visualize_prediction(model, val_dataset, index=15)
```

```

/tmp/ipykernel_1008/3756778289.py:33: UserWarning: Glyph 128225 (\N{SATELLITE ANTENNA}) missing from font(s) DejaVu Sans.
plt.tight_layout()
/tmp/ipykernel_1008/3756778289.py:33: UserWarning: Glyph 128506 (\N{WORLD MAP}) missing from font(s) DejaVu Sans.
plt.tight_layout()
/tmp/ipykernel_1008/3756778289.py:33: UserWarning: Glyph 129302 (\N{ROBOT FACE}) missing from font(s) DejaVu Sans.
plt.tight_layout()

```



 Part 8: Saving Your Trained Model Training takes time. Now that the model has learned the weights to detect floods, we need to save this state to a file (.pth). We will then trigger a download so you have a local copy to plug into your backend server.

```

from google.colab import files

# Define the file name for your saved model
model_save_path = '/content/flood_unet_resnet34.pth'

# Save only the state_dict (the learned weights), which is the PyTorch best practice
torch.save(model.state_dict(), model_save_path)
print(f"✅ Model weights saved to {model_save_path}")

# Trigger the browser to download the file to your actual computer
print("📄 Downloading model to your local machine...")
files.download(model_save_path)

```

✅ Model weights saved to /content/flood_unet_resnet34.pth
 📄 Downloading model to your local machine...

 Part 9: The Dashboard Bridge (Pixels to GPS) Your web dashboard cannot read a 2D array of pixels; it needs GeoJSON polygons (GPS coordinates) to draw the flood on a map. This function demonstrates how your future backend will take the model's prediction, read the spatial metadata from the original satellite file, and convert the flood pixels into real-world latitude and longitude boundaries.

```

import rasterio.features
from shapely.geometry import shape

def extract_flood_polygons(prediction_mask, original_tif_path):
    # 1. Open the original satellite image just to get its coordinate mapping (Transform)
    with rasterio.open(original_tif_path) as src:
        transform = src.transform
        crs = src.crs # Coordinate Reference System

    # 2. Make sure the mask is formatted properly for extraction (numpy integer array)
    mask = prediction_mask.astype('uint8')

    # 3. Extract shapes (polygons) where the mask is 1 (Flood)
    # rasterio.features.shapes returns generator of (geojson_polygon_dict, value)
    polygons = []
    for geom, value in rasterio.features.shapes(mask, transform=transform):
        if value == 1.0: # Only grab the flood zones
            # Convert to a Shapely geometry object for easy handling later
            poly = shape(geom)

```

```

polygons.append(geom) # Storing the raw GeoJSON dict

print(f"📄 Extracted {len(polygons)} distinct flooded regions as GeoJSON polygons.")
print(f"📍 Map Coordinate System: {crs}")

return polygons

# Let's test it on the 10th image we visualized earlier
# Let's test it on the 10th image from the validation set
sample_num = 15

# Get the original index from the full dataset
subset_index = val_dataset.indices[sample_num]

# Get the image path and true mask
test_image_path = full_dataset.image_paths[subset_index]
_, true_mask = full_dataset[subset_index]

# Extract coordinates
gps_polygons = extract_flood_polygons(
    true_mask.numpy().squeeze(),
    test_image_path
)

# Show the raw GeoJSON data for the first polygon
if gps_polygons:
    print("\nSample GeoJSON Output (Ready for your Mapbox/Leaflet frontend):")
    print(gps_polygons[0])

```

📄 Extracted 87 distinct flooded regions as GeoJSON polygons.
 📍 Map Coordinate System: EPSG:4326

Sample GeoJSON Output (Ready for your Mapbox/Leaflet frontend):
 {'type': 'Polygon', 'coordinates': [[[105.88478186528165, 12.183850198513069), (105.88478186528165, 12.183760366984657), (105.88

Double-click (or enter) to edit

```

import rasterio
from google.colab import files

# 1. Grab the 10th image from the dataset you already initialized in Part 4
# We know Image #10 is a good flood example from your visualization
# Use the same sample number that gave you the good prediction
sample_num = 20 # <-- change this to whatever sample worked for you

subset_index = val_dataset.indices[sample_num]
real_image_path = full_dataset.image_paths[subset_index]

print(f"✅ Extracting real professional SAR data from: {real_image_path}")

# 2. Open and save it as a clean local file
with rasterio.open(real_image_path) as src:
    data = src.read()
    meta = src.meta

# Save it to the Colab 'hard drive'
with rasterio.open('real_flood_test.tif', 'w', **meta) as dst:
    dst.write(data)

# 3. Trigger the download to your computer
print("📄 Downloading 'real_flood_test.tif' to your machine...")
files.download('real_flood_test.tif')

```

✅ Extracting real professional SAR data from: /content/sen1floods11/HandLabeled/S1Hand/India_166709_S1Hand.tif
 📄 Downloading 'real_flood_test.tif' to your machine...